

SQL INTERVIEW QUESTIONS

What is SQL?

- It stands for Structured Query Language. A programming language used for interaction with relational database management systems (RDBMS). This includes fetching, updating, inserting, and removing data from tables.

What types of SQL commands (or SQL subsets) do you know?

- **Data Definition Language (DDL)** – to define and modify the structure of a database.
- **Data Manipulation Language (DML)** – to access, manipulate, and modify data in a database.
- **Data Control Language (DCL)** – to control user access to the data in the database and give or revoke privileges to a specific user or a group of users.
- **Transaction Control Language (TCL)** – to control transactions in a database.
- **Data Query Language (DQL)** – to perform queries on the data in a database to retrieve the necessary information from it.

Give some examples of common SQL commands of each type.

- **DDL:** CREATE, ALTER TABLE, DROP, TRUNCATE, and ADD COLUMN
- **DML:** UPDATE, DELETE, and INSERT
- **DCL:** GRANT and REVOKE
- **TCL:** COMMIT, SET TRANSACTION, ROLLBACK, and SAVEPOINT
- **DQL:** – SELECT

Difference Between Delete,Drop,Truncate?

1)DELETE

- Purpose: Removes rows from a table based on a specified condition.
- Syntax: DELETE FROM table_name WHERE condition;
- Effect: Deletes only the data rows, leaving the table structure intact.
- DELETE FROM employees WHERE department = 'Sales';

- When you use DELETE without a condition, it indeed removes all rows from the table, but the table structure (including columns, constraints, etc.) remains intact

2)DROP

- Completely removes a table or database from the database system.
- Syntax: DROP TABLE table_name; or DROP DATABASE database_name;
- Effect: Deletes the entire table structure, including all rows, constraints, and triggers.
- DROP TABLE employees;

3) TRUNCATE

- Removes all rows from a table, but keeps the table structure intact.
- Syntax: TRUNCATE TABLE table_name;
- Effect: Deletes all rows quickly and resets any auto-increment counters.
- TRUNCATE TABLE employees;

What is RDBMS? Give some examples of RDBMS.

- It stands for Relational Database Management System. It's the most common type of DBMS used for working with data stored in multiple tables related to each other by means of shared keys. The SQL programming language is particularly designed to interact with RDBMS. Some examples of RDBMS are MySQL, PostgreSQL, Oracle, MariaDB, etc.

What are tables and fields in SQL?

- A table is an organized set of related data stored in a tabular form, i.e., in rows and columns. A field is another term for a column of a table.

What is a constraint, and why use constraints?

- A set of conditions defining the type of data that can be input into each column of a table. Constraints ensure data integrity in a table and block undesired actions.

What SQL constraints do you know?

- DEFAULT – provides a default value for a column.

- UNIQUE – allows only unique values.
- NOT NULL – allows only non-null values.
- PRIMARY KEY – allows only unique and strictly non-null values (NOT NULL and UNIQUE).
- FOREIGN KEY – provides shared keys between two and more tables.

What is a join?

A clause used to combine and retrieve records from two or multiple tables. SQL tables can be joined based on the relationship between the columns of those tables.

What types of joins do you know?

- (INNER) JOIN – returns only those records that satisfy a defined join condition in both (or all) tables. It's a default SQL join.
- LEFT (OUTER) JOIN – returns all records from the left table and those records from the right table that satisfy a defined join condition.
- RIGHT (OUTER) JOIN – returns all records from the right table and those records from the left table that satisfy a defined join condition.
- FULL (OUTER) JOIN – returns all records from both (or all) tables. It can be considered as a combination of left and right joins.

Difference Between SQL JOIN and Subquery?

Subquery	JOIN
A subquery is a query nested inside another query and is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.	A JOIN is a means for combining fields from two tables by using values common to each.
Subqueries can be slower than JOINS, especially if the subquery returns a large number of rows.	JOINS are generally faster than subqueries, especially for large datasets.
Subqueries can be more complex and harder to read, especially when there are multiple levels of nesting.	JOINS can be easier to read and understand, especially for simple queries.
Subqueries can be used in SELECT, WHERE, and FROM clauses, offering more flexibility.	JOINS are used to combine rows from two or more tables based on a related column.
Subqueries are often used when the result of the query is not known or dynamic.	JOINS are used when the relationships between the tables are known and fixed.

What is a primary key?

- A column (or multiple columns) of a table to which the PRIMARY KEY constraint was imposed to ensure unique and non-null values in that column. In other words, a primary key is a combination of the NOT NULL and UNIQUE constraints. The primary key uniquely identifies each record of the table. Each table should contain a primary key and can't contain more than one primary key.

What is a unique key?

- A column (or multiple columns) of a table to which the UNIQUE constraint was imposed to ensure unique values in that column, including a possible NULL value (the only one).

What is a foreign key?

- A column (or multiple columns) of a table to which the FOREIGN KEY constraint was imposed to link this column to the primary key in another table (or several tables). The purpose of foreign keys is to keep connected various tables of a database.

Unique key vs Primary Key?

Unique key

- A unique key is a column or set of columns that prevents duplicate values in a column and can store one null value.

Primary key

- A primary key is a unique identifier for a row that cannot contain any null values or duplicate values

What is an index?

- critical technique used in databases to improve the speed of data retrieval operations. It works by creating a data structure that allows the database to quickly locate specific rows in a table without scanning the entire table.
- **Unique index** – doesn't allow duplicates in a table column and hence helps maintain data integrity.

- **Clustered index** – defines the physical order of records of a database table and performs data searching based on the key values. A table can have only one clustered index.
- **Non-clustered index** – keeps the order of the table records that doesn't match the physical order of the actual data on the disk. It means that the data is stored in one place and a non-clustered index – in another one. A table can have multiple non-clustered indexes.

What is a schema?

- A collection of database structural elements such as tables, stored procedures, indexes, functions, and triggers. It shows the overall database architecture, specifies the relationships between various objects of a database, and defines different access permissions for them.

What types of SQL operators do you know?

- **Arithmetic** (+, -, *, /, etc.)
- **Comparison** (>, <, =, >=, etc.)
- **Compound** (+=, -=, *=, /=, etc.)
- **Logical** (AND, OR, NOT, BETWEEN, etc.)
- **String** (% , _ , + , ^ , etc.)
- **Set** (UNION, UNION ALL, INTERSECT, and MINUS (or EXCEPT))

Union vs Union All?

- The UNION operator in SQL is used to combine the results of two or more SELECT queries into a single result set. It eliminates duplicate rows by default.
- If you want to include duplicate rows, you can use UNION ALL

SELECT column1, column2, ...

FROM table1

UNION

SELECT column1, column2, ...

FROM table2;

What is an alias?

- A temporary name given to a table (or a column in a table) while executing a certain SQL query. Aliases are used to improve the code readability and make the code more compact. An alias is introduced with the AS keyword:
- SELECT col_1 AS column FROM table_name;

What are some common statements used with the SELECT query?

- A condition imposed on a SQL query to filter the data to obtain the desired result. Some examples are WHERE, LIMIT, HAVING, LIKE, AND, OR, ORDER BY, etc.

Difference between Having and Where Clause?

1. WHERE Clause

- Purpose: Filters rows before any grouping or aggregation is performed.
- Usage: Applied to individual rows in the table.
- Used in queries without aggregation.
- SELECT *
- FROM employees
- WHERE department = 'Sales';

2) HAVING Clause

- Purpose: Filters groups of rows after the GROUP BY clause has been applied, usually in conjunction with aggregate functions.
- Usage: Applied to groups of rows, typically after aggregation has been performed.
- Used in queries with aggregation and GROUP BY.

SELECT department, COUNT(*)

FROM employees

GROUP BY department

HAVING COUNT(*) > 10;

In many queries, both WHERE and HAVING are used together:

```
SELECT department, COUNT(*)
```

```
FROM employees
```

```
WHERE status = 'Active'
```

```
GROUP BY department
```

```
HAVING COUNT(*) > 5;
```

SQL Partition Clause?

- The OVER clause in SQL is used with window functions to perform calculations across a set of table rows that are related to the current row. This is particularly useful for running totals, ranking, or calculating moving averages, without needing to group rows like in an aggregate function (GROUP BY).
- A PARTITION BY clause is used to partition rows of table into groups. It is useful when we have to perform a calculation on individual rows of a group using other rows of that group.
- It is always used inside OVER() clause.
- The partition formed by partition clause are also known as Window.
- This clause works on windows functions only. Like- RANK(), LEAD(), LAG() etc.
- If this clause is omitted in OVER() clause, then whole table is considered as a single partition.

- `Window_function ( expression )`
- `Over ( partition by expr [order clause] [frame_clause] )`

Consider, a table "Hacker":

h_id	h_name	challenge_id	score
3	shubh	111	20
2	aayush	111	80
5	krithik	112	40
5	krithik	114	90
4	tushar	112	30
1	parth	112	40

We have to find the rank of hackers in each challenge. That means we have to list all participated hackers of a challenge along with their rank in that challenge.

-

Query:

```
select challenge_id, h_id, h_name, score,  
       dense_rank() over ( partition by challenge_id order by score desc )  
       as "rank", from hacker;
```

Explanation:

In the above query, ***partition by*** clause will partition table into groups that are having same challenge_id.

order by will arrange the hackers of each partition in descending order by “scores”.

over() clause defines how to partition and order rows of table, which is to be processed by window function rank().

dense_rank() is a window function, which will assign rank in ordered partition of challenges. If two hackers have same scores then they will be assigned same rank.

Output:



challenge_id	h_id	h_name	score	rank
111	2	aayush	80	1
111	3	shubh	20	2
112	5	krithik	40	1
112	1	parth	40	1
112	4	tushar	30	2
114	5	krithik	90	1

Thus, we get list of all hackers along with their ranks in the individual challenges.

How to create a table?

- Using the CREATE TABLE statement. For example, to create a table with three columns of predefined datatypes, we apply the following syntax:

```
CREATE TABLE table_name (col_1 datatype,  
                           col_2 datatype,  
                           col_3 datatype);
```

How to update a table?

- Using the UPDATE statement. The syntax is:

```
UPDATE table_name  
SET col_1 = value_1, column_2 = value_2  
WHERE condition;
```

How to get the count of records in a table?

- Using the COUNT() aggregate function with the asterisk passed as its argument
- SELECT COUNT(*) FROM table_name;

How to sort records in a table?

- Using the ORDER BY statement:

```
SELECT * FROM table_name  
ORDER BY col_1;
```

- We can specify that we need a descending order using the DESC keyword; otherwise, the order will be ascending by default. Also, we can sort by more than one column and specify for each one, ascending or descending order separately. For example:

```
SELECT * FROM table_name  
ORDER BY col_1 DESC, col_3, col_6 DESC;
```

How to select all columns from a table?

Using the asterisk * with the SELECT statement. The syntax is: SELECT * FROM table_name;.

How to select common records from two tables?

- Using the INTERSECT statement:

```
SELECT * FROM table_1
INTERSECT
SELECT * FROM table_1;
```

What is the DISTINCT statement and how do you use it?

- This statement is used with the SELECT statement to filter out duplicates and return only unique values from a column of a table. The syntax is:

```
SELECT DISTINCT col_1
FROM table_name;
```

What is NULL value? How is it different from zero or a blank space?

- A NULL value indicates the absence of data for a certain cell of a table. Instead, zero is a valid numeric value, and an empty string is a legal string of zero length.

What are relationships? Give some examples.

- Relationships are the connections and correlations between entities, basically meaning how two or more tables of a database are related to one another.
2. **One-to-One (1:1) Relationship** : Each record in Table A is related to exactly one record in Table B, and vice versa.
 - *Usage Example*: A table of employees and a table of employee_passports, where each employee has exactly one passport and each passport is assigned to exactly one employee.
 3. **One-to-Many** : A single record in Table A is related to multiple records in Table B. However, each record in Table B is related to only one record in Table A.
 - A table of departments and a table of employees, where each department can have multiple employees, but each employee belongs to only one department.
 4. **Many-to-One** : multiple records in Table A are associated with a single record in Table B.
 - many courses taught by single teacher

5. **Many-to-Many** : Records in Table A can be related to multiple records in Table B, and records in Table B can be related to multiple records in Table A.
- A table of students and a table of courses, where each student can enroll in multiple courses, and each course can have multiple students enrolled.

What is a function in SQL, and why use functions?

A database object representing a set of SQL statements frequently used for a certain task. A function takes in some input parameters, performs calculations or other manipulations on them, and returns the result. Functions help improve code readability and avoid repetition of the same code snippets.

- **Aggregate functions** – work on multiple, usually grouped records for the provided columns of a table, and return a single value (usually by group).
- **Scalar functions** – work on each individual value and return a single value.

What aggregate functions do you know?

- **AVG()** – returns the average value
- **SUM()** – returns the sum of values
- **MIN()** – returns the minimum value
- **MAX()** – returns the maximum value
- **COUNT()** – returns the number of rows, including those with null values
- **FIRST()** – returns the first value from a column
- **LAST()** – returns the last value from a column

What scalar functions do you know?

- **LEN()** (in other SQL flavors – **LENGTH()**) – returns the length of a string, including the blank spaces
- **UCASE()** (in other SQL flavors – **UPPER()**) – returns a string converted to the upper case
- **LCASE()** (in other SQL flavors – **LOWER()**) – returns a string converted to the lower case

- INITCAP() – returns a string converted to the title case (i.e., each word of the string starts from a capital letter)
- MID() (in other SQL flavors – SUBSTR()) – extracts a substring from a string
- ROUND() – returns the numerical value rounded to a specified number of decimals
- NOW() – returns the current date and time

What are case manipulation functions? Give some examples.

Case manipulation functions represent a subset of character functions, and they're used to change the case of the text data. With these functions, we can convert the data into the upper, lower, or title case.

- UCASE() (in other SQL flavors – UPPER()) – returns a string converted to the upper case
- LCASE() (in other SQL flavors – LOWER()) – returns a string converted to the lower case
- INITCAP() – returns a string converted to the title case (i.e., each word of the string starts from a capital letter)

What is the difference between local and global variables?

- Local variables can be accessed only inside the function in which they were declared. Instead, global variables, being declared outside any function, are stored in fixed memory structures and can be used throughout the entire program.

What is the default data ordering with the ORDER BY statement, and how do you change it?

- By default, the order is ascending. To change it to descending, we need to add the DESC keyword as follows:

```
SELECT * FROM table_name
```

```
ORDER BY col_1 DESC;
```

What operator is used in the query for pattern matching?

- The LIKE operator in combination with the % and _ wildcards. The % wildcard represents any number of characters including zero, while _ – strictly one character.

What is a composite primary key?

- The primary key of a table, based on multiple columns.

Normalization and Denormalization?

- **Normalization** is the process of organizing a database to reduce redundancy and improve data integrity. It involves dividing a database into two or more tables and defining relationships between them to minimize duplication and ensure data dependencies make sense.
- **Denormalization** is the process of intentionally introducing redundancy into a database by merging tables or adding redundant data. This is usually done to optimize read performance by reducing the complexity of joins and speeding up query responses.

redundancy refers to the repetition of the same data in multiple locations within a database

Insertion Anomaly: Occurs when certain attributes cannot be inserted into the database without the presence of other attributes. For example, if you cannot add a new department to the table without also adding a new employee, it's an insertion anomaly.

Update Anomaly: Happens when updating data in one place requires updating it in multiple places to maintain consistency. For example, if an employee's department head is changed, it must be updated in all records where the employee's data appears.

Deletion Anomaly: Occurs when the deletion of data results in unintended loss of other data. For instance, if deleting an employee's record also deletes department information, it is a deletion anomaly.

First Normal Form (1NF):

Requirement: The table must have a primary key, and each column must contain atomic (indivisible) values. No repeating groups or arrays.

Second Normal Form (2NF):

Requirement: The table must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key (i.e., no partial dependencies on a subset of a composite key).

Example: If you have a table where a composite primary key consists of `student_id` and `course_id`, and there's an attribute like `course_name` that depends only on `course_id`, then `course_name` should be moved to a separate table.

Third Normal Form (3NF):

Requirement: The table must be in 2NF, and all the attributes must be directly dependent on the primary key (i.e., no transitive dependencies).

Example: If a table has `student_id`, `department`, and `department_head`, and `department_head` depends on `department` rather than `student_id`, then `department_head` should be moved to a separate table.

Examples of Normalization

1. 1NF Example:

- Before:

StudentID	Name	Courses
1	Alice	Math, Science
2	Bob	Math, English

- After 1NF:

StudentID	Name	Course
1	Alice	Math
1	Alice	Science
2	Bob	Math
2	Bob	English

2. 2NF Example:

- Before:

StudentID	CourseID	CourseName	Instructor
1	101	Math	Dr. Smith
1	102	Science	Dr. Jones
2	101	Math	Dr. Smith

- After 2NF:

- Courses Table:

CourseID	CourseName	Instructor
101	Math	Dr. Smith
102	Science	Dr. Jones

- Enrollments Table:

StudentID	CourseID
1	101
1	102
2	101



3. 3NF Example:

- Before:

StudentID	Name	Department	DepartmentHead
1	Alice	Math	Dr. White
2	Bob	Science	Dr. Black

- After 3NF:

- Students Table:

StudentID	Name	Department
1	Alice	Math
2	Bob	Science

- Departments Table:

Department	DepartmentHead
Math	Dr. White
Science	Dr. Black

What is the difference between renaming a column and giving an alias to it?

- Renaming a column means permanently changing its actual name in the original table. Giving an alias to a column means giving it a temporary name while executing an SQL query, with the purpose to make the code more readable and compact.

What is the CASE() function?

- The way to implement the *if-then-else* logic in SQL. This function sequentially checks the provided conditions in the WHEN clauses and returns the value from the corresponding THEN clause when the first condition is satisfied. If none of the conditions is satisfied, the function returns the value from the ELSE clause in case it's provided, otherwise, it returns NULL. The syntax is:

CASE

WHEN condition_1 THEN value_1

WHEN condition_2 THEN value_2

WHEN condition_3 THEN value_3

...

ELSE value

END;

How do you add a record to a table?

- Using the INSERT INTO statement in combination with VALUES. The syntax is:

INSERT INTO table_name

VALUES (value_1, value_2, ...);

How to add a column to a table?

- Using the ALTER TABLE statement in combination with ADD. The syntax is:

ALTER TABLE table_name

ADD column_name datatype;

How to rename a column of a table?

- Using the ALTER TABLE statement in combination with RENAME COLUMN ... TO ... The syntax is:

ALTER TABLE table_name


```
RENAME COLUMN old_column_name TO new_column_name;
```

How to delete a column from a table?

- Using the ALTER TABLE statement in combination with DROP COLUMN. The syntax is:

```
ALTER TABLE table_name
```

```
DROP COLUMN column_name;
```

How to select all even or all odd records in a table?

- By checking the remainder of the division by 2. In some SQL versions (e.g., PostgreSQL and My SQL), we use the MOD function, in the others (Microsoft SQL Server and SQLite) – the modulo operator (%). To select all even records using MOD:

```
SELECT * FROM table_name
```

```
WHERE MOD(ID_column, 2) = 0;
```

To select all even records using %:

```
SELECT * FROM table_name
```

```
WHERE ID_column % 2 = 0;
```

- To select all odd records, the syntax is identical in both cases, only that **we would use the inequality operator <> instead of =.**

How to insert many rows in a table?

```
INSERT INTO table_name
```

```
VALUES (value_1, value_2, ...),
```

```
(value_3, value_4, ...),
```

```
(value_5, value_6, ...),
```

```
...;
```

How to find the nth highest value in a column of a table?

- Using the OFFSET clause. For example, to find the 6th highest value from a column, we would use the following syntax:
SELECT * FROM table_name

```
ORDER BY column_name DESC
LIMIT 1
OFFSET 5;
```

How to find the values in a text column of a table that start with a certain letter?

- Using the LIKE operator in combination with the % and _ wildcards. For example, we need to find all surnames in a table that start with "A". The query is:

```
SELECT * FROM table_name
WHERE surname LIKE 'A_';
```

How to find the last id in a table?

- Using the MAX() function. Otherwise, in many SQL versions, we can use the following syntax:

```
SELECT id
FROM table_name
ORDER BY id DESC
LIMIT 1;
```

How to select random rows from a table?

- Using the RAND() function in combination with ORDER BY and LIMIT. In some SQL flavors, such as PostgreSQL, it's called RANDOM(). For example, the following code will return five random rows from a table in MySQL:

```
SELECT * FROM table_name
ORDER BY RAND()
LIMIT 5;
```

Database Transaction Cycle

- sequence of operations performed in a database to ensure that a series of SQL statements are executed reliably and consistently.

BEGIN TRANSACTION

- A transaction starts with the **beginning** of an operation, either explicitly (using BEGIN TRANSACTION) or implicitly when the first SQL statement is executed.

EXECUTE SQL QUERIES

- In this step, SQL queries are executed within the scope of the transaction. Any changes made to the database are **not visible** to other users until the transaction is committed.
- **Operations:** Multiple SQL queries like INSERT, UPDATE, DELETE, or SELECT are executed as part of the transaction. These operations are temporarily stored in memory.

INTERMEDIATE STATE: SAVEPOINTS (Optional)

- **SAVEPOINTS** can be created during the transaction to set a rollback point to recover only a part of the transaction instead of the entire transaction.
- **Operation:** Savepoints allow partial rollbacks, helping developers control the rollback of a portion of the transaction.

COMMIT OR ROLLBACK

At the end of the transaction, there are two possible actions: either the transaction is **committed** (making all changes permanent) or **rolled back** (undoing all changes).

a) COMMIT

- **COMMIT** finalizes the transaction, making all changes permanent in the database.
- **Operation:** The system writes all changes to disk, ensuring that they are saved even if the database crashes afterward. Other users can now see the changes.

b) ROLLBACK

- **ROLLBACK** cancels the transaction, discarding all changes made during the transaction and restoring the database to its state before the transaction began.
- **Operation:** If an error or issue occurs, you can undo all changes made during the transaction by issuing a ROLLBACK.

ROLLBACK TO SAVEPOINT

- **Operation:** If you have created SAVEPOINTS during the transaction, you can roll back to a specific savepoint instead of the entire transaction.

TRANSACTION END

- After the COMMIT or ROLLBACK, the transaction ends, and the system is ready for the next transaction.
- **Operation:** The system clears the temporary transaction data, resets transaction locks, and prepares for the next transaction.

Triggers

A **trigger** is a special kind of stored procedure that is automatically executed in response to certain events (e.g., INSERT, UPDATE, or DELETE) on a table. Triggers are used to enforce data integrity rules, audit changes, or implement complex business logic automatically.

Types of Triggers:

1. **Before Trigger:** Executes before the actual INSERT, UPDATE, or DELETE operation is carried out.
2. **After Trigger:** Executes after the actual operation.
3. **Instead of Trigger:** Used in cases where the action can be replaced with a custom operation (common in views).

```
CREATE TRIGGER update_balance
```

```
BEFORE UPDATE ON accounts
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF :NEW.balance < 0 THEN
```

```
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds');
```

```
    END IF;
```

```
END;
```

In this example, a trigger is created to ensure that an account cannot be updated to have a negative balance.

Checkpoint

A **checkpoint** is a process in a database system where all the data changes in memory (also known as the "transaction log") are written to the physical database file on disk. It's primarily used in crash recovery scenarios.

- **Purpose of a Checkpoint:**

- Reduces recovery time during a database crash by ensuring that a certain amount of work is already written to the database.
- Clears the transaction log by flushing dirty pages (pages in memory that have been modified but not yet written to disk).

- **When a Checkpoint Occurs:**

- Automatically by the database at intervals, depending on workload.
- Manually by issuing a CHECKPOINT command.

Conclusion

The **transaction cycle** ensures that a sequence of operations either completes entirely or has no effect at all, providing a robust mechanism for handling database consistency and error recovery. Here's a quick breakdown:

1. **Begin Transaction:** Start the transaction.
2. **Execute Queries:** Perform various SQL operations.
3. **Use SAVEPOINTS (Optional):** Create intermediate rollback points.
4. **Commit or Rollback:** Finalize the transaction or undo all changes.
5. **Transaction End:** The transaction completes, ensuring data consistency.

ACID PROPERTIES

1. Atomicity (All or Nothing)

- **Meaning:** A transaction/all operations of transaction must happen completely or not at all. If any part of the transaction fails, the whole transaction is canceled, and no changes are made.
- **Example:** If you're transferring money between two bank accounts:
 - **Success:** Money is deducted from one account and added to another.
 - **Failure:** If the money is deducted but not added to the other account, the whole transaction is undone, and no money is moved.

2. Consistency (Valid Data)

- **Meaning:** The database must remain in a valid state before and after the transaction. A transaction cannot leave the database in an incorrect or broken state.

- **Example:** In the bank transfer example, the total balance of the two accounts before and after the transfer should always add up to the same amount.

In the context of the bank transfer, when we say "**the total balance of the two accounts before and after the transfer should always add up to the same amount**", it means that:

- The **total combined** money in both accounts **before** the transfer should be the **same as the total after** the transfer.
- The individual balances of each account will change, but the total amount of money in the system remains the same. No money is lost or created during the transfer.

Example:

Let's walk through a simple transfer:

1. **Before the Transfer:**

- Account A has \$500.
- Account B has \$300.
- Total money in the system: $\$500 + \$300 = \$800$.

2. **Transfer \$100 from A to B:**

- **Deduct \$100 from Account A:** Account A now has \$400.
- **Add \$100 to Account B:** Account B now has \$400.

3. **After the Transfer:**

- Account A has \$400.
- Account B has \$400.
- Total money in the system: $\$400 + \$400 = \$800$.

Key Point:

- **Before transfer:** Total = \$800 (A's balance + B's balance).
- **After transfer:** Total = \$800 (A's balance + B's balance).

3. Isolation (No Interference)

- **Meaning:** Transactions must be independent of each other. One transaction shouldn't affect another while it's still running.
 - **Example:** If two people try to withdraw money from the same account at the same time, each withdrawal should be processed one after the other, not simultaneously, to avoid incorrect balances.
-

4. Durability (Permanent Changes)

- **Meaning:** Once a transaction is committed, its changes are saved permanently. Even if the system crashes afterward, the changes will remain.
 - **Example:** After transferring money, if the system crashes, the transaction will still be complete, and the money transfer will remain successful.
-

Summary Example

Imagine transferring \$100 from Account A to Account B:

1. **Atomicity:** If any step of the transfer fails, no money is moved.
2. **Consistency:** Before and after the transfer, the total of both accounts remains the same.

Example:

- **Before the transfer:**
 - Account A has \$500.
 - Account B has \$300.
 - Total balance of both accounts: $\$500 + \$300 = \$800$.
- **After transferring \$100 from A to B:**
 - Account A now has \$400 ($\$500 - \100).
 - Account B now has \$400 ($\$300 + \100).
 - The total balance of both accounts is still $\$400 + \$400 = \$800$.

So, the **total amount in both accounts combined** remains the same (\$800), but the individual balances change. This consistency ensures that no money is lost or created during the transaction.

3. **Isolation:** If someone else is also making transfers at the same time, each transaction is handled one by one.
4. **Durability:** Once the transfer is complete, the system ensures that even if it crashes, the money stays transferred.

These properties help keep the database accurate and reliable during operations.

SQL TIME FUNCTIONS:-

1. CURDATE()

- **Description:** Returns the current date (without the time).
- `SELECT CURDATE();` -- Returns '2024-10-02' if today is October 2, 2024

2. CURTIME()

- **Description:** Returns the current time.
- `SELECT CURTIME();` -- Returns '14:23:01' for 2:23:01 PM

3. NOW()

- **Description:** Returns the current date and time
- `SELECT NOW();` -- Returns '2024-10-02 14:23:01'

4. DATE()

- **Description:** Extracts the date part of a DATETIME or TIMESTAMP.
- `SELECT DATE(NOW());` -- Returns '2024-10-02'

5. DATEDIFF(date1, date2)

- **Description:** Returns the number of days between two dates.
- `SELECT DATEDIFF('2024-12-25', '2024-10-02');` -- Returns 84 (days between the two dates)

6. DATE_ADD()

- **Description:** Adds a specified interval to a date.
- `SELECT DATE_ADD('2024-10-02', INTERVAL 5 DAY);` -- Returns '2024-10-07'

7. DATE_SUB()

- **Description:** Subtracts a specified interval from a date.
- `SELECT DATE_SUB('2024-10-02', INTERVAL 5 DAY);` -- Returns '2024-09-27'

8. EXTRACT()

- **Description:** Extracts a part of a date or time.
- `SELECT EXTRACT(YEAR FROM '2024-10-02');` -- Returns 2024

9. TIMESTAMPDIFF(unit, datetime1, datetime2)

- **Description:** Returns the difference between two dates in the specified unit (e.g., SECOND, MINUTE, HOUR, DAY, etc.).
- `SELECT TIMESTAMPDIFF(DAY, '2024-10-02', '2024-12-25');` -- Returns 84

10. ADDDATE()

- **Description:** Adds a specified number of days to a date.
- `SELECT ADDDATE('2024-10-02', 10);` -- Returns '2024-10-12'

11. SUBDATE()

- **Description:** Subtracts a specified number of days from a date.
- `SELECT SUBDATE('2024-10-02', 10);` -- Returns '2024-09-22'

12. YEAR(), MONTH(), DAY()

- **Description:** Extracts the year, month, or day from a date.
- `SELECT YEAR('2024-10-02');` -- Returns 2024
- `SELECT MONTH('2024-10-02');` -- Returns 10
- `SELECT DAY('2024-10-02');` -- Returns 2

13. DAYOFWEEK()

- **Description:** Returns the weekday index (1 = Sunday, 7 = Saturday).
- `SELECT DAYOFWEEK('2024-10-02');` -- Returns 4 (Wednesday)

14. STR_TO_DATE()

- **Description:** Converts a string to a date using a specified format.
- `SELECT STR_TO_DATE('02-10-2024', '%d-%m-%Y');` -- Returns '2024-10-02'